

Basic Security Audit of a Sample Website

Overview:

For this task, we conducted a hands-on security audit of **OWASP Juice Shop**, a deliberately vulnerable web application designed for practicing security testing. We set up a local instance using Docker Desktop, then performed manual vulnerability testing using browser DevTools to identify and confirm real security flaws across multiple categories, including Cross-Site Scripting, Broken Authentication, and Sensitive Data Exposure. Each finding was reproduced with specific payloads and verified through screenshots documenting the exploit and its outcome. Then supplemented the manual testing with an automated scan using **OWASP ZAP** to check for additional issues like missing security headers and misconfigurations. For each vulnerability identified, we documented how it was found, its potential real-world impact, and a practical remediation recommendation.

OWASP Juice Shop:

We set up **OWASP Juice Shop** locally using **Docker Desktop** on a Windows machine. After enabling WSL2 and Virtual Machine Platform to support Docker, we pulled the official Juice Shop image and ran it as a container, exposing it on localhost:3000 for testing.

```
PS C:\WINDOWS\system32> docker pull bkimminich/juice-shop
Using default tag: latest
latest: Pulling from bkimminich/juice-shop
d6b1b89eccac: Pull complete
3214acf345c0: Pull complete
47de5dd0b812: Pull complete
bdfd7f7e5bf6: Pull complete
8224d91da70b: Pull complete
ebddc55facdc: Pull complete
1a4be5562d92: Pull complete
dd64bf2dd177: Pull complete
```

Figure 1 Docker pull Juice shop Image

```
Digest: sha256:e68144772ebaaca0ec11b38d44903af92416793230288ef7c5437fc4f26850a
Status: Downloaded newer image for bkimminich/juice-shop:latest
docker.io/bkimminich/juice-shop:latest
PS C:\WINDOWS\system32> docker run -d -p 3000:3000 bkimminich/juice-shop
4200b4616fdcf12660ff6622436d4e58e0b3531eea9292762e41a8c586c6b093
PS C:\WINDOWS\system32> █
```

Figure 2 Ran image as Container

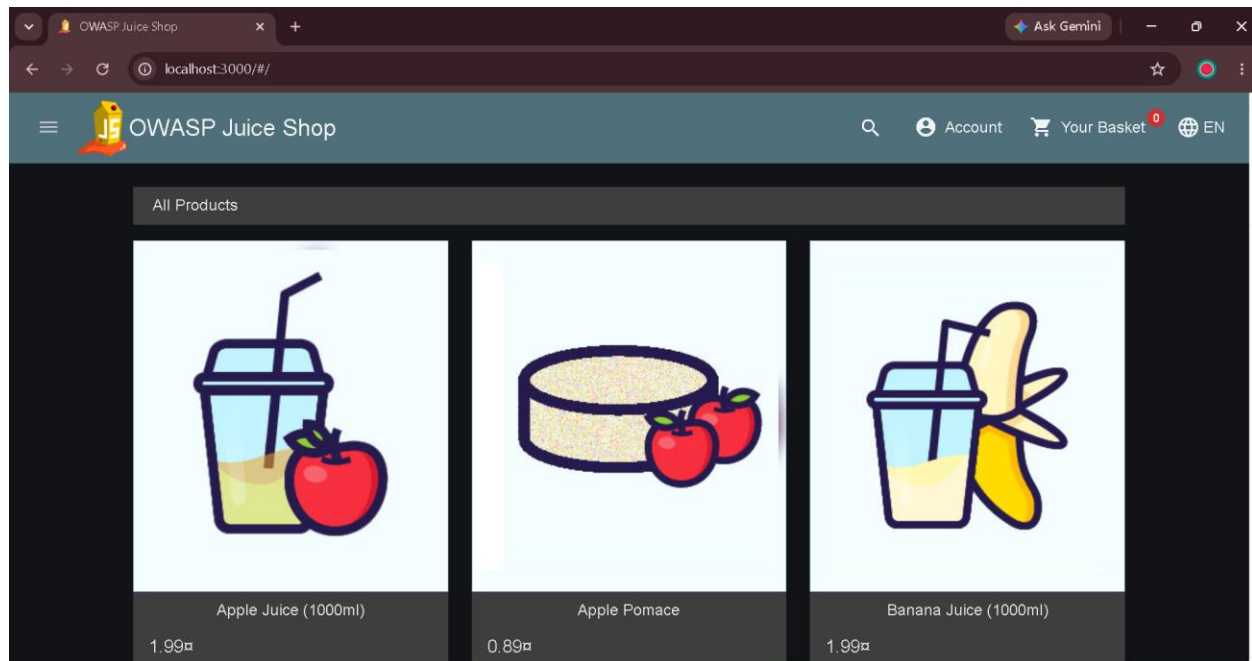


Figure 3 OWASP Juice shop Dashboard

Manual Testing:

Manual testing involved actively probing the application's inputs and authentication flow with crafted payloads to identify exploitable vulnerabilities. Unlike ZAP's passive scan, this **active scan** approach directly triggered and confirmed real security flaws, including XSS execution and a full authentication bypass.

Finding 1: Reflected XSS:

- **Title:** Reflected Cross-Site Scripting (XSS) in Search Function
- **Where it is:** Search bar
- **Category:** XSS (Injection-based client-side vulnerability)
- **How found:** Entered `<img src=x onerror=alert('XSS')>` into the search bar

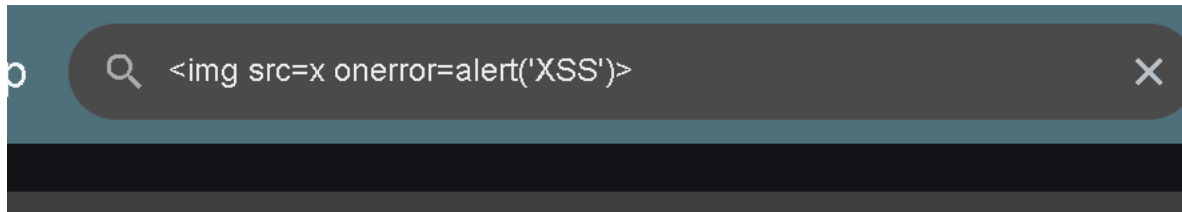
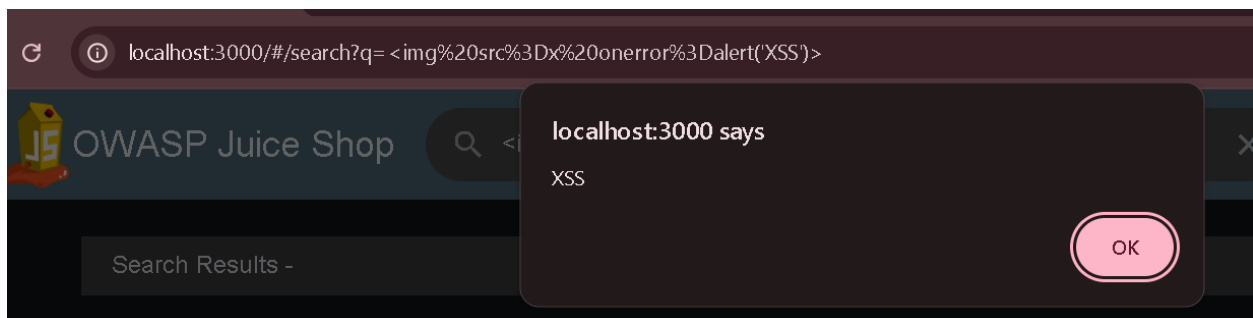


Figure 4 String in search bar

- **Evidence:** Alert popup displaying "XSS" fired immediately.



- **Impact:** An attacker could craft a malicious search URL (like the one visible in your address bar) and send it to a victim via email/chat. If the victim clicks it while logged in, the attacker's script could steal session cookies, redirect them to a phishing page, or perform actions on their behalf.
- **Remediation:** Sanitize and encode all user input before rendering it back into the page (e.g., HTML-encode special characters like <, >, "). Implement a strict Content Security Policy (CSP) as defense-in-depth.

Finding 2: SQL Injection:

- **Title:** SQL Injection Authentication Bypass
- **Where it is:** Login Form
- **Category:** Broken Authentication / Injection
- **How found:** Entered ' OR 1=1-- in the email field with any password on the login form

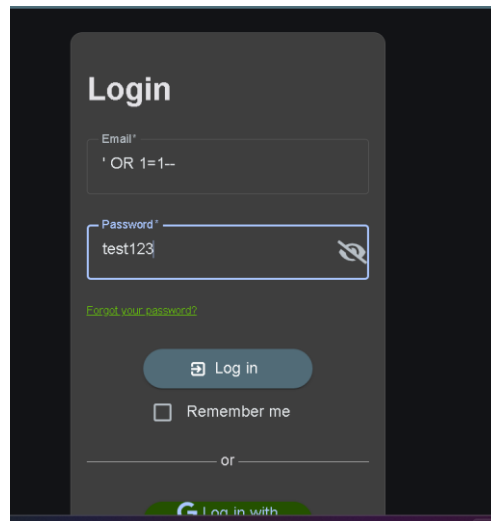


Figure 5 Login form

- **Evidence:** App confirmed "Login Admin" challenge solved; logged in as administrator without valid credentials

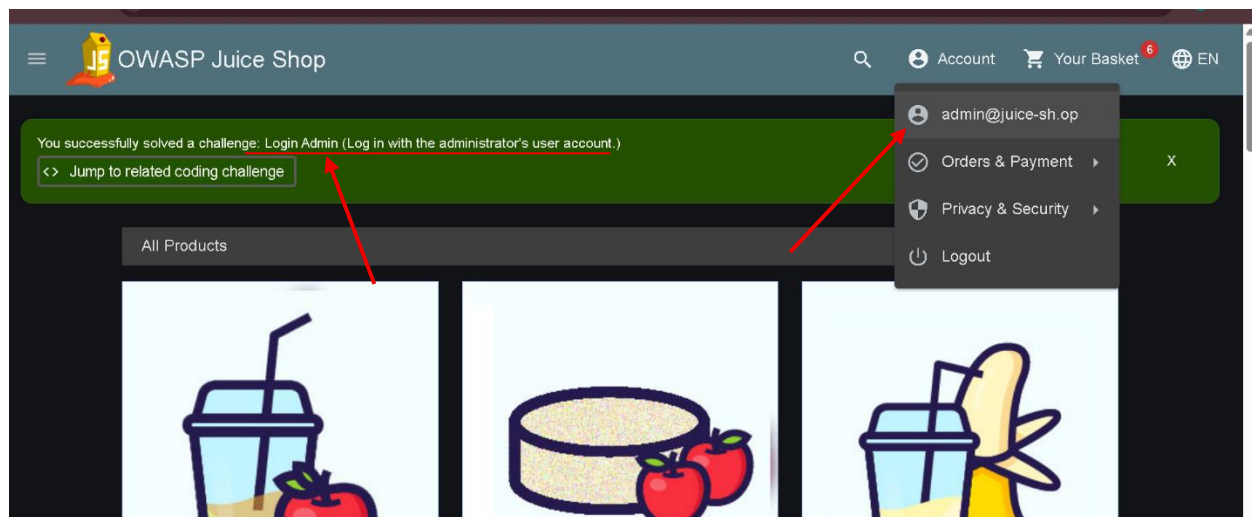


Figure 6 Logged in as Administrator

- **Impact:** Complete authentication bypass — an attacker can log in as *any* user, including administrators, without knowing any password. This could lead to full account takeover, access to admin functionality, and exposure of all user data.
- **Remediation:** Use parameterized queries / prepared statements for all database access instead of building SQL strings via concatenation. Additionally, apply input validation and use an ORM that escapes input by default.

Finding 3: Data Exposure:

- **Title:** Sensitive Data Exposure via /rest/user/whoami API Endpoint

- **Where it is:** whoami endpoint.
- **Category:** Sensitive Data Exposure / Security Misconfiguration
- **How found:** Opened DevTools → Network tab while browsing the Basket page, inspected the whoami XHR request's Response body
- **Evidence:** Response returned internal id, email, lastLoginIp, and profileImagePath, none of which are rendered or needed on that page

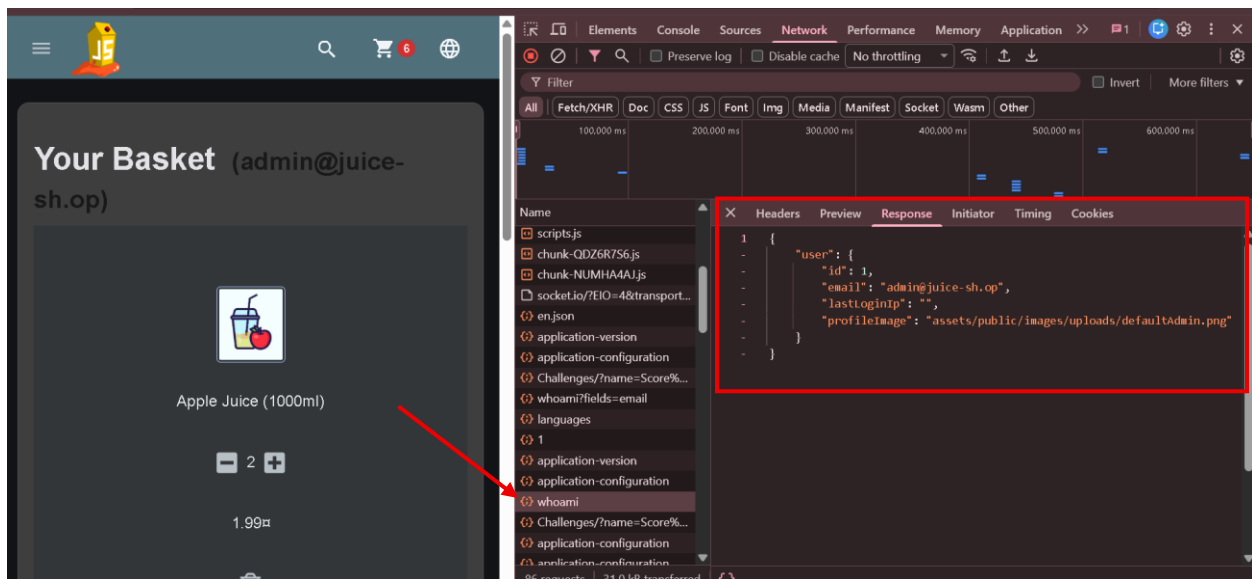


Figure 7 Response revealing Sensitive data

- **Impact:** Exposes internal user identifiers and metadata that could aid an attacker in enumerating accounts, correlating user IDs across other endpoints, or building a profile of a user for further attacks (e.g. IDOR attacks using the exposed id). While not catastrophic alone, it violates the principle of least data exposure and often compounds with other vulnerabilities.
- **Remediation:** Enforce field-level response scoping server-side for all API endpoints (not just ones with an explicit ?fields= parameter), return only the data a specific frontend view needs, rather than the full user object by default. Apply the principle of least privilege to API design across the board.

OWASP ZAP:

To complement manual testing, we ran **OWASP ZAP's automated scan** against the local Juice Shop instance. ZAP's spider crawled the application to map its structure, then performed a **passive scan**, analyzing traffic and response headers without actively attacking the app, surfacing issues like missing security headers and configuration weaknesses that complement the actively exploited vulnerabilities found through manual testing.

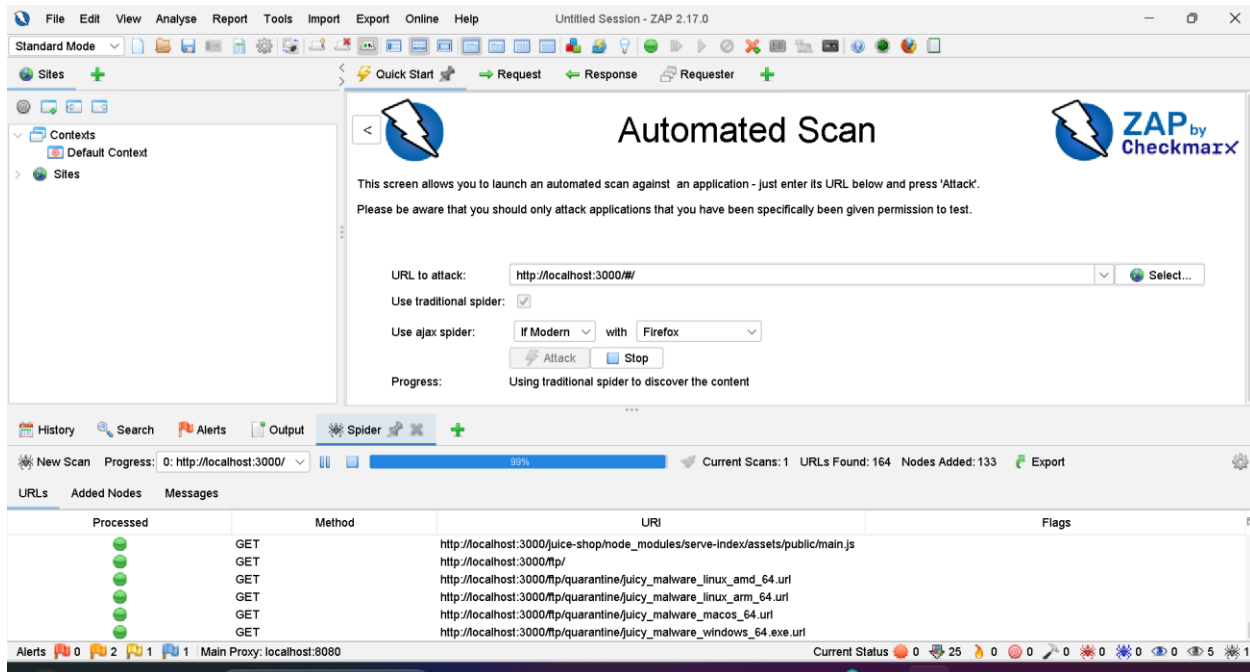


Figure 8 OWASP ZAP scanning website

Finding 4: Content Security Policy (CSP) header not set:

- **Category:** Security Misconfiguration
- **How Found:** Identified via OWASP ZAP's passive scan while crawling `http://localhost:3000/`
- **Evidence:** ZAP flagged the response as missing a Content-Security-Policy header (Risk: Medium, Confidence: High, CWE-693)
- **Impact:** Without a CSP, the browser has no additional layer of defense against XSS or data injection attacks. This directly compounds the reflected XSS vulnerability found manually in Finding 1 — a properly configured CSP could have blocked the injected script from executing even if input sanitization failed.

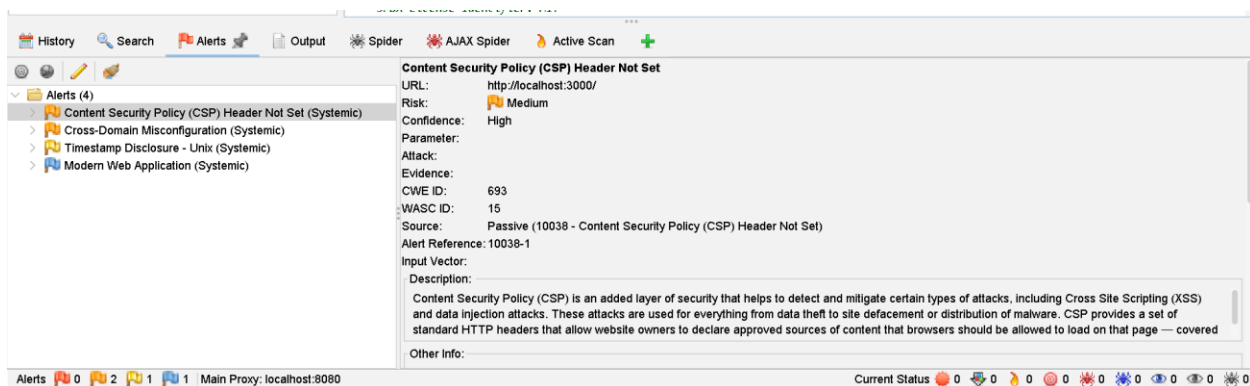
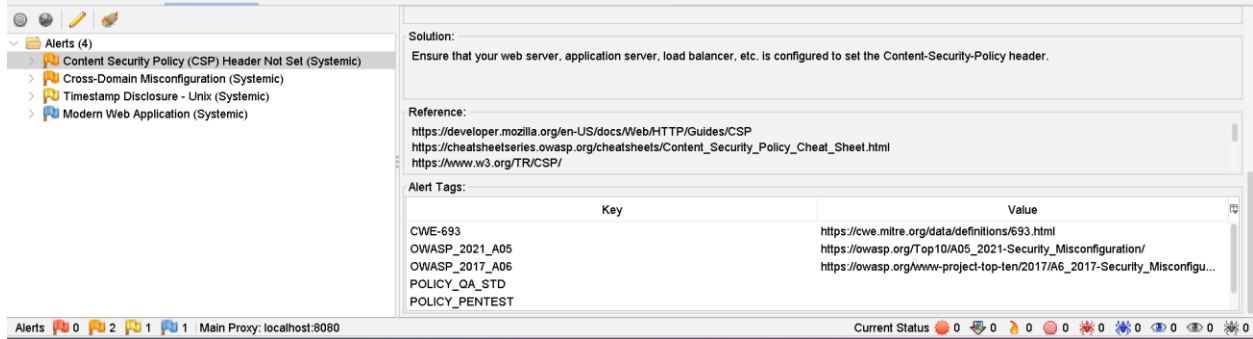


Figure 9 Alert Details



Finding 5: Cross-Domain Misconfiguration (CORS):

- **Category:** Security Misconfiguration / Broken Access Control
- **How Found:** Identified via OWASP ZAP's passive scan on <http://localhost:3000/chunk-VS3A3LTT.js>
- **Evidence:** Response header Access-Control-Allow-Origin: * was flagged (Risk: Medium, Confidence: Medium, CWE-264)
- **Impact:** An overly permissive CORS policy allows arbitrary third-party domains to make cross-origin requests to unauthenticated APIs on this server, potentially enabling attackers to pull data via a malicious site if a victim's browser interacts with it.

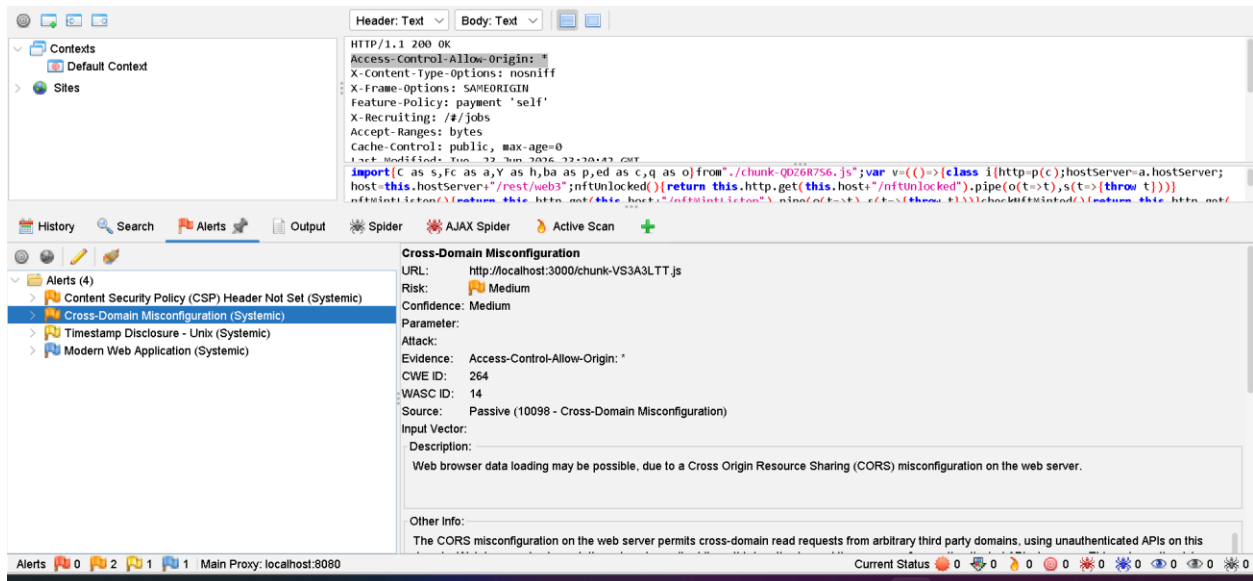


Figure 10 Cross-Domain Misconfiguration Details

Note: Other 2 findings by OWASP ZAP were Information Disclosure (Low-severity).